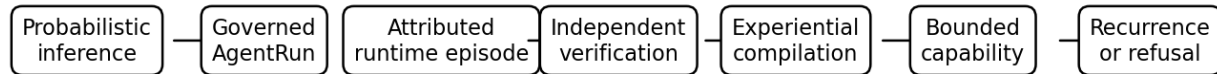


PROOF-GOVERNED EXPERIENTIAL COMPILATION

Transforming Verified Agentic Inference into Bounded Reusable Compute

BEAST Working Paper v1.0 - Initial Full Draft



Continuing proof, policy compatibility, authority bounds, and state fingerprints govern every transition.

Figure 1. The proof-governed experiential compilation pipeline.

Byron Bunt

EdgeK-BEAST Research Initiative

20 July 2026

CLAIM-BOUNDED SYSTEMS PREPRINT

This draft is grounded in the supplied BEAST source archive, Python test estate, benchmark outputs, closure receipts, and the earlier Inference Economy Inversion working paper. It distinguishes directly observed results, local proof-of-mechanism evidence, environment-gated mechanisms, and future hypotheses.

Abstract

Large language model agents are generally treated as producers of transient responses: a task is submitted, probabilistic inference is consumed, and the resulting output is accepted, repaired, or discarded. Even where traces are retained, they commonly serve as conversational memory, demonstrations, or retrieval material rather than independently governed computational capability. This paper introduces proof-governed experiential compilation (PGEC), a systems approach through which successful agentic software-execution episodes are observed, attributed, verified, parameterised, and compiled into bounded reusable capabilities. The approach treats the complete governed runtime episode, rather than the prompt or response alone, as the source unit of experience. An episode binds task state, context provenance, provider activity, tool use, authority consumption, isolated mutation, verifier outcomes, resource costs, and resulting physical effects. Successful episodes may be normalised into candidate capabilities with explicit preconditions, parameters, execution graphs, postconditions, applicability boundaries, verifier requirements, rollback semantics, and maximum authority. Promotion is allowed only after independent verification, and recurrence is permitted only while repository, policy, execution-target, and behavioural fingerprints remain compatible. Otherwise, the correct outcome is refusal, revalidation, or escalation. A staged BEAST evidence programme supplies proof of mechanism. In a paired 24-observation slice, the governed lane achieved 24/24 verified and hidden-test completions compared with 2/24 in the comparison lane, while using fewer cloud calls in 12 pairs and more in none. Six task families first reached deterministic zero-call recurrence at occurrence three. Additional evidence reported 12/12 provider-independent capability consumptions, mutation-triggered blocking of six stale reuses with six verified recoveries and zero false reuse, repeated crystal-only held-out proposals with zero generated tokens, and a 12-case hostile-refusal closure with no physical effects in refused cases. These results do not establish universal autonomous software competence or general inference replacement. They demonstrate that, for bounded task families, probabilistic agent experience can be converted into attributable, authority-constrained, independently verifiable local capability whose validity depends on continuing proof.

Keywords: agentic AI; runtime verification; experiential compilation; proof-carrying capability; inference economy; crystallized compute; capability security; evidence; safe refusal; software agents

1. Introduction

Agentic software systems are commonly evaluated by whether they produce a correct patch, invoke tools, or pass a benchmark. This outcome-oriented view is necessary but incomplete. It does not fully answer who performed an action, under what authority, against which state, through which physical processes, with what evidence, or whether the verified work can safely reduce later inference demand.

The prevailing operational model treats model inference as rented cognition. Each task, even when structurally repetitive, initiates another probabilistic call. Prompt caches and key-value cache systems reduce repeated token computation, while agent memories retrieve prior text or examples. These mechanisms improve serving efficiency but do not, by themselves, establish that a prior software action remains behaviourally valid, authorised, and safe under the current repository and policy state.

BEAST investigates a stronger possibility. A successful agent episode may leave behind more than a transcript. When the episode is durably identified, physically attributed, independently verified, stripped of incidental details, parameterised, and bound to an explicit authority envelope, it may become a reusable local capability. This transformation is called proof-governed experiential compilation.

PGEC is not a claim that probabilistic inference becomes deterministic merely because it succeeded once. It is a controlled conversion process. The original episode supplies evidence from which a candidate capability may be extracted. The candidate remains subordinate to applicability tests, verifier requirements, policy,

revocation, and human-controlled promotion. A compiled capability therefore contains both positive competence, describing what it may perform, and negative competence, describing when it must refuse, revalidate, or escalate.

The central research claim is bounded: for defined task families, verified agentic experience can be transformed into a reusable computational object that safely displaces later inference only when continuing proof shows that the current task remains inside the promoted capability boundary.

1.1 Contributions

- A formal definition of proof-governed experiential compilation as the conversion of attributable, verified agent runtime episodes into bounded reusable computational capabilities.
- A runtime-episode model that binds initial state, task identity, context provenance, tool actions, authority consumption, process and effect evidence, verifier outcomes, and resource economics.
- A capability model containing preconditions, parameters, execution graph, postconditions, applicability predicate, verifier set, maximum authority, and decay or revocation rules.
- A proof-of-mechanism synthesis grounded in BEAST Python modules, tests, benchmark records, and closure receipts, including deterministic recurrence, cross-provider consumption, mutation-sensitive refusal, failed-closed adapter evidence, capsule enforcement, and hostile refusal.
- A governance argument that safe refusal and human-controlled promotion are constitutive parts of capability, rather than external safety decorations.
- An economic interpretation in which inference economy inversion is the downstream consequence of experiential compilation: probabilistic compute is spent where uncertainty remains and preserved as local governed infrastructure where proof permits.

1.2 Research questions

RQ	Question
RQ1	Can a completed agentic task be represented as an attributable runtime episode containing sufficient state, action, authority, verification, and effect evidence for later capability extraction?
RQ2	Can repeated verified episodes be compiled into a parameterised reusable capability rather than retained only as transcripts or cached responses?
RQ3	Can the compiled capability displace later probabilistic inference while preserving visible and hidden verification outcomes?
RQ4	Can repository, policy, tool-schema, target, or behavioural drift be detected before stale execution, causing safe refusal, revalidation, or escalation?
RQ5	Can a capability originating from one provider-mediated episode be consumed independently of that provider because its authority derives from evidence rather than provider identity?
RQ6	Can experiential compilation preserve bounded authority, human-controlled promotion, auditability, and rollback during recurrence?

2. Conceptual Framework

2.1 From inference conversion to experiential compilation

The earlier BEAST Inference Economy Inversion paper introduced inference conversion: each probabilistic call can produce positive capability, negative capability, friction evidence, counterfactual hypotheses, escrow

records, temporal forks, or durable intelligence artifacts. It defined crystallized compute as the durable local product of this conversion. PGEC sharpens that theory by identifying the missing transformation mechanism. Crystallization names a promoted state; experiential compilation names the evidence-governed process by which runtime experience is converted into that state.



Continuing proof, policy compatibility, authority bounds, and state fingerprints govern every transition.

Figure 2. PGEC converts a governed runtime episode into a capability, while proof and policy remain active at every transition.

2.2 Definitions

Term	Definition
Agent runtime episode	An append-only, attributable representation of a governed task from initial state through actions, authority use, verification, and resulting effects.
Experiential evidence	Observed state, tool, process, file, socket, approval, verifier, and resource records generated by an episode; not hidden model reasoning.
Candidate capability	A parameterised execution proposal extracted from one or more successful episodes, with no inherent right to execute or promote.
Promoted capability / crystal	A candidate that has passed defined evidence, applicability, authority, and verifier gates and is eligible for bounded recurrence.
Positive competence	The transformations that a capability may perform inside its proven boundary.
Negative competence	The conditions under which the capability must refuse, revalidate, demote, or escalate.
Continuing proof	Current evidence that repository state, policy, tool schema, target identity, and required postconditions remain compatible with promotion evidence.
Safe recurrence	Execution that remains inside the capability boundary and independently re-establishes required postconditions.
Safe refusal	A zero-effect decision not to execute when applicability, identity, evidence, authority, or policy requirements fail.

2.3 Formal model

A governed runtime episode is represented as:

$$E = (S_0, T, C, M, A, G, V, S_1, R) \quad (1)$$

where S_0 is initial state; T the task envelope; C context provenance; M model/provider activity; A consumed authority; G the observed execution and causal graph; V verifier evidence; S_1 resulting state; and R resource and economic evidence.

Compilation over one or more episodes produces a candidate capability:

$$K = \mathcal{C}(E_1, E_2, \dots, E_n) = (P, \Theta, G_k, Q, \Gamma, \mathcal{V}, \mathcal{A}, \Delta) \quad (2)$$

P denotes preconditions; Θ parameters; G_k the bounded execution graph; Q postconditions; Γ an applicability predicate; $\mathcal{V}$ the verifier set; $\mathcal{A}$ the maximum authority envelope; and Δ decay, demotion, and revocation rules.

Execution is admissible only when current state, task, policy, and execution target satisfy applicability, and requested authority is a subset of the promoted authority envelope:

$$\Gamma(S, T, \Pi, X) = 1 \wedge A_{\text{requested}} \subseteq \mathcal{A} \quad (3)$$

After execution, verifier agreement must re-establish the required postcondition:

$$\mathcal{V}(S', Q) = 1 \quad (4)$$

If applicability fails, the correct system outcome is one of three governed non-execution paths:

$$\Gamma(S, T, \Pi, X) = 0 \Rightarrow \text{REFUSE} \vee \text{REVALIDATE} \vee \text{ESCALATE} \quad (5)$$

The economic value of recurrence can be expressed without treating estimates as realised savings:

$$V_K = C_{\text{avoided}} - (C_{\text{lookup}} + C_{\text{verify}} + C_{\text{risk}} + C_{\text{maintenance}}) \quad (6)$$

A capability has positive operational value only when avoided inference cost exceeds lookup, verification, residual risk, and maintenance cost. The equation prevents “zero-call” from becoming an unquestioned objective: after material drift, fresh inference and verification may be economically and ethically correct.

2.4 Core propositions

Experiential Compilation Proposition.

For a bounded task family, an agentic execution episode may be converted into reusable local computation when the system can identify invariant causal structure, parameterise incidental variation, bind execution to explicit preconditions and authority, and independently verify that recurrence preserves required postconditions.

Proof Preservation Principle.

A compiled capability remains valid only while its current applicability evidence is at least as strong as the evidence under which it was promoted.

Promotion Sovereignty Principle.

Autonomous work may proceed inside an isolated mission boundary, but transition into the authoritative operator workspace remains a distinct, human-controlled promotion act.

3. BEAST Implementation Substrate

The supplied full build contains approximately 14,059 files, including a large kernel, test estate, benchmark archive, evidence records, and closure bundles. The analysis in this draft does not infer implementation from class names alone. It uses named source modules together with tests and result artifacts as implementation and proof anchors.

3.1 Durable agent runtime

The canonical runtime package includes `run_engine.py`, `run_state.py`, `run_events.py`, `run_store.py`, `run_cancel.py`, `planner_runtime.py`, `tool_runtime.py`, `worktree_tools.py`, and `promotion_engine.py`. Route-level tests exercise run creation, execution, and tool runtime, while phase-specific tests cover worktree mutation, SourcePlan handoff, evidence crystallization, approvals, and capability consumption.

Runtime concern	Representative Python implementation	Representative tests
Durable run state and events	<code>app/kernel/agents/run_engine.py</code> ; <code>run_state.py</code> ; <code>run_events.py</code> ; <code>run_store.py</code>	<code>tests/test_agent_run_store.py</code> ; <code>tests/routes/test_agent_run_routes.py</code> ; <code>tests/routes/test_agent_run_execution.py</code>
Model-directed planning and tools	<code>planner_runtime.py</code> ; <code>tool_runtime.py</code> ; <code>tool_registry.py</code> ; <code>worktree_tools.py</code>	<code>tests/test_agent_planner_runtime.py</code> ; <code>tests/routes/test_agent_tool_runtime.py</code> ; <code>tests/phase2/test_worktree_mutation_tools.py</code>
Promotion boundary	<code>promotion_engine.py</code> ; <code>app/kernel/evidence/sourceplan_handoff.py</code> ; <code>post_apply_gate.py</code>	<code>tests/phase3/test_sourceplan_handoff.py</code> ; <code>tests/test_sourceplan_evidence.py</code>
Durable approvals and one-use authority	<code>app/kernel/evidence/operator_approval.py</code> ; <code>app/kernel/integration/one_use_capability.py</code>	<code>tests/phase3/test_operator_approval.py</code> ; <code>tests/phase4/test_phase4_6_request_bound_capability.py</code>
Evidence and compatibility	<code>evidence_ledger.py</code> ; <code>fingerprint_engine.py</code> ; <code>compatibility_engine.py</code> ; <code>equivalence_engine.py</code>	<code>tests/phase3/test_evidence_ledger.py</code> ; <code>tests/test_replay_evidence_binding.py</code>
Crystallization and recurrence	<code>runtime_crystallizer.py</code> ; <code>governed_crystal_executor.py</code> ; <code>crystal_reuse_gateway.py</code>	<code>tests/test_runtime_crystallizer.py</code> ; <code>tests/test_governed_crystal_executor.py</code> ; <code>tests/test_crystal_reuse_gateway.py</code>

3.2 Runtime episodes and Sensorium

Sensorium modules and tests provide the observation substrate for process, socket, file, and physical-effect evidence. The design separates chronological sequence from causality: temporal adjacency can order events, but causal edges require explicit evidence such as producer-consumer relationships, resource identity, process lineage, or verified state transition. This distinction prevents a transcript from being mistaken for proof.

Representative implementations include sensorium runtime and journal components, `capsule_sensorium.py`, `unified_evidence_packet.py`, process and socket integration, and physical-effect tests. The corresponding test suite includes `test_sensorium_contracts.py`, `test_sensorium_journal.py`, `test_sensorium_runtime.py`, `test_sensorium_socket_integration.py`, `test_sensorium_physical_effects.py`, and `test_sensorium_crystallization.py`.

3.3 Typed crystals, capsules, and authority

The capability substrate includes `typed_crystal_ir.py`, `typed_crystal_interpreter.py`, `crystal_candidate_adapter.py`, `crystal_generalizer.py`, `crystal_hypergraph.py`, `governed_crystal_executor.py`,

sealed_capsule.py, crystal_bus.py, crystal_runtime_boundary.py, and capsule enforcement modules. Tests cover typed interpretation, generalization, compound composition, hostile production matrices, sealed capsules, runtime boundaries, and physical lifecycle.

This architecture makes a vital distinction: a vector, similarity score, or retrieved trace may nominate a candidate, but it cannot confer authority. Execution requires compatibility, policy, verifier, and capability checks. Promotion remains separately governed.

4. Method

4.1 Study design

The study is a claim-bounded systems investigation combining controlled paired observations, staged benchmark evidence, negative tests, source-level implementation inspection, and closure receipts. It is not a completed general benchmark of autonomous coding ability. The earlier master package contains a 198-record non-IID staged analysis set and a designed 450-row matrix; inferential statistics are restricted to the mature 24-pair NVIDIA slice where pairing assumptions hold.

4.2 Evidence classes

Class	Meaning	Examples used in this paper
Live paired evidence	Actual provider-mediated paired observations with comparable task instances.	24-pair mature NVIDIA slice; cross-provider reuse cases.
Controlled local proof	Deterministic or local mechanisms demonstrating a bounded property.	Crystal-only held-out route; mutation ladder; capsule-required canary.
Negative/hostile evidence	Deliberate violations expected to fail closed with zero unintended effects.	False reuse, stale lease, wrong peer, missing seals, workspace escape.
Implementation evidence	Python modules and tests establishing that mechanisms are present and exercised.	AgentRun, Sensorium, Worktree Forge, SourcePlan, crystal runtime, approvals.
Blueprint/future work	Designed but not yet sufficiently demonstrated mechanisms.	Universal natural crystallization, general multi-crystal composition, broad cross-host portability.

4.3 Metrics

Verified completion requires visible and/or hidden verifier success according to the task contract. Quality-preserving cloud-call displacement (QPCCD) is defined for paired cases as:

$$QPCCD_i = 1 / [(Q_C, i \geq Q_B, i) \wedge (Calls_C, i < Calls_B, i)] \quad (7)$$

False reuse is a redline event in which a reused capability disagrees with the verified baseline or required postcondition. Refusal correctness requires the expected refusal category, zero unintended physical effects, and no residual side effects.

$$FRR = FalseReuse / ReuseAttempts \quad (8)$$

$$RCR = CorrectRefusals / HostileCases \quad (9)$$

4.4 Statistical treatment

For the 24 paired NVIDIA observations, the earlier paper used McNemar tests for paired binary verified and hidden-test outcomes, an exact one-sided sign test for cloud-call counts, and a Wilson confidence interval for the QPCCD proportion. The broader staged and closure evidence is reported descriptively because independence and sampling assumptions are not satisfied.

5. Results

5.1 Behaviour-preserving recurrence

In the mature NVIDIA slice, the comparison lane passed hidden tests in 2 of 24 paired observations, while the governed BEAST lane passed 24 of 24. The reported McNemar continuity-corrected statistic was 20.05 with $p < .001$. The same 2-versus-24 pattern applied to verified completion. This result supports proof of mechanism that the governed system preserved or improved verified behaviour within this bounded slice.

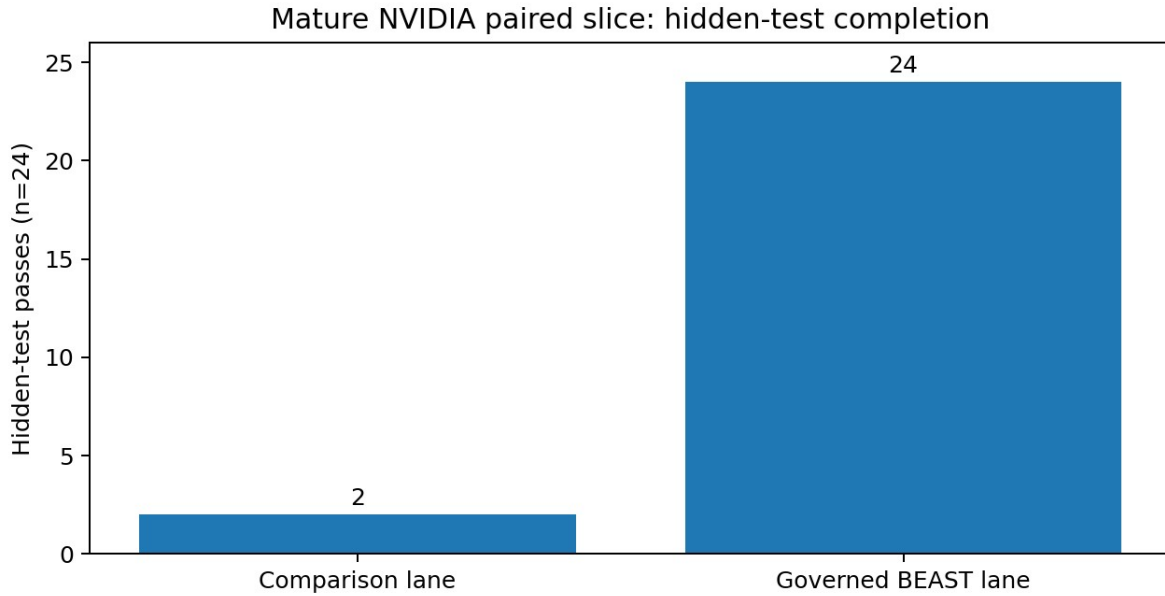


Figure 3. Hidden-test completion in the mature paired slice. Source: mature NVIDIA evidence summarised in the v0.9 working paper.

The governed lane used fewer cloud calls in 12 pairs, tied in 12, and used more in none. The exact one-sided sign test reported $p = 0.000244$ after excluding ties. QPCCD was $12/24 = 0.50$, with a 95% Wilson interval of $[0.314, 0.686]$.

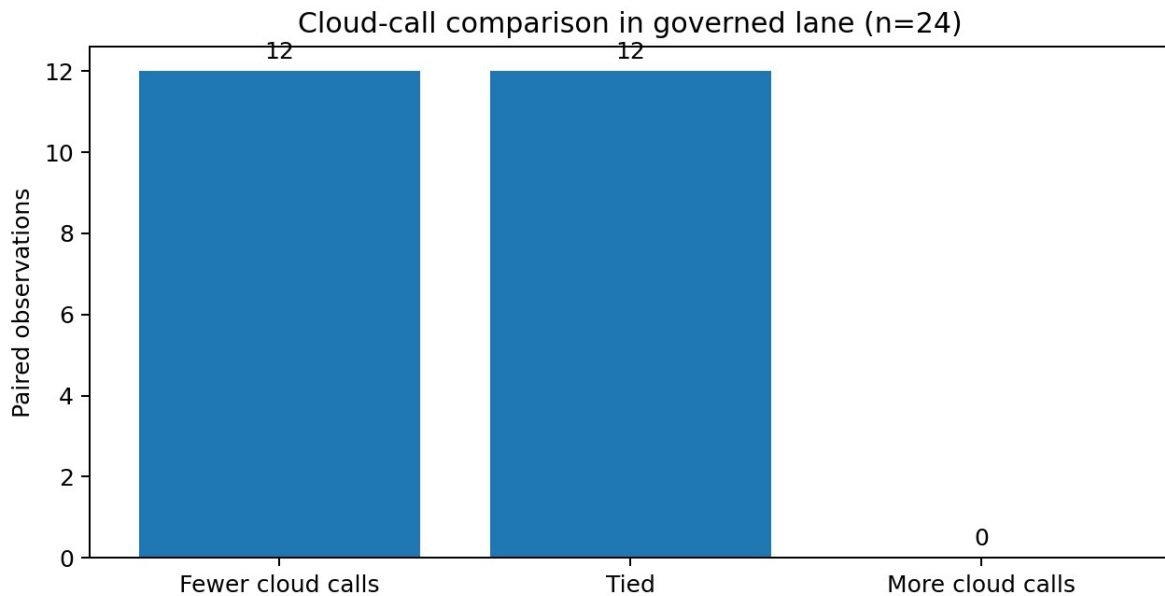


Figure 4. Cloud-call comparison across paired observations.

5.2 Evidence accumulation and transition to recurrence

All six task families first reached deterministic zero-call recurrence at occurrence three and retained hidden-test success. The consistency across schema validation, provider alias normalisation, patch compilation, syntax checking, route diagnostics, and secret redaction suggests that occurrence three reflected the evidence-accumulation policy rather than one unusually easy family. This is evidence of a controlled transition: inference was initially used to resolve uncertainty; after sufficient verified repetition, the system admitted a bounded deterministic path.

5.3 Provider-independent capability consumption

The master evidence reported 12/12 cross-provider primary reuse cases under Mistral and Cohere route identities, with zero target-provider execution, zero cloud calls, preserved visible and hidden behaviour, zero incorrect reuse, and an estimated 18,768 avoided tokens. The result supports the proposition that the reusable object is not the original provider response. Authority for recurrence derives from repository-bound evidence, fingerprints, and verifiers rather than source-provider identity.

5.4 Mutation-sensitive refusal and recovery

The occurrence-10 mutation package was deliberately a safety-recovery experiment rather than a displacement experiment. Six stale reuses were blocked after material drift, all six cases recovered through fresh governed execution, and false reuse remained zero. The wider mutation ladder distinguished four tiers. Cosmetic changes allowed six local reuses; semantic-adjacent and structural or tool-schema changes each triggered six shadow revalidations; breaking target or test changes triggered six escalations. Across 24 policy decisions, incorrect reuse remained zero.

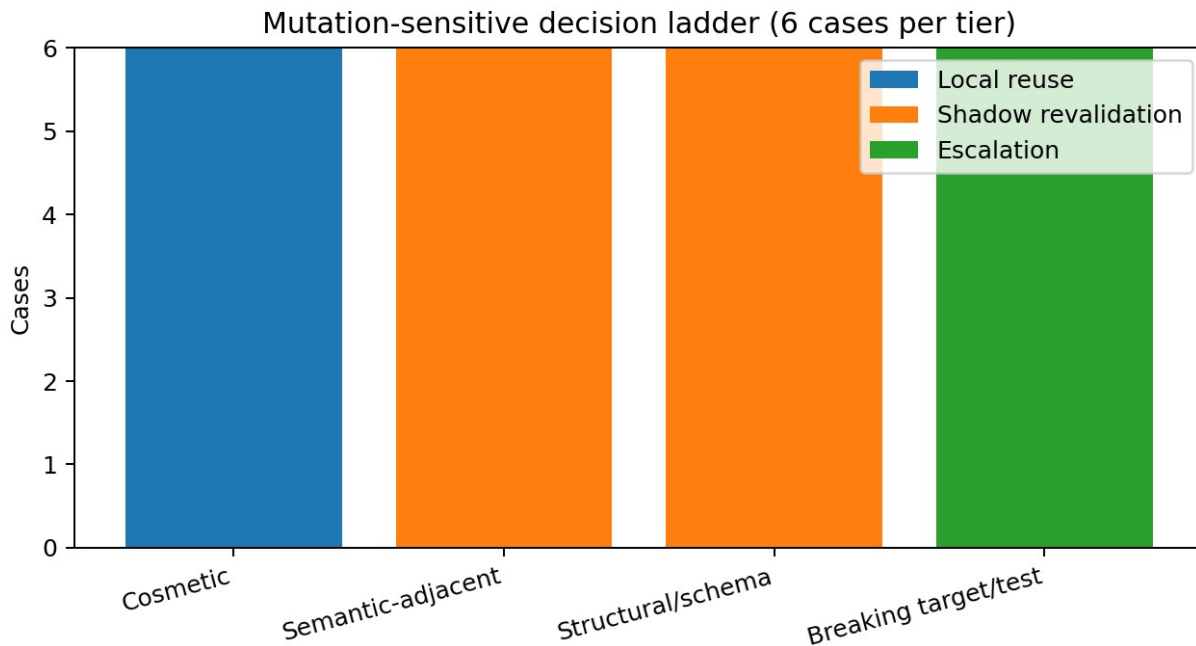


Figure 5. Mutation-sensitive decision ladder. Reuse is not the objective when proof has been disturbed.

This result is central to PGEC. A compiled capability is not valuable merely because it runs without a model. It is valuable because it knows when prior proof no longer authorises recurrence.

5.5 Crystal-only recurrence and failed-closed adapter evidence

In the Phase 8 held-out proposal gauntlet, the crystal-only route generated schema-valid, hidden-verifier-passing proposals for 6/6 tasks with zero generated tokens and approximately 0.020 ms average latency. A

follow-up gauntlet reproduced 6/6 at approximately 0.022 ms. By contrast, an incompatible loaded adapter based on a diagnostic random-initialised tiny GPT-2 runtime generated unusable non-JSON output and failed all governance checks. It remained unpromoted. The evidence therefore supports deterministic recurrence and failed-closed governance, but not general weight-level LoRA improvement.

5.6 Capsule-required execution closure

The Grand Closure G8 receipt tested a canary task class under capsule-required enforcement. A lawful verified capsule produced one physical effect. Missing and unverified capsules were refused with zero physical effects; no legacy fallback was used. An unrelated task class remained on its prior path. This demonstrates that the system can make the compiled execution representation mandatory for a bounded class without silently falling back to an unverified legacy path.

Case	Expected/observed outcome	Physical effects	Fallback
Lawful verified capsule	capsule_success	1	No
Missing capsule	capsule_required_refusal	0	No
Unverified capsule	capsule_required_refusal	0	No
Unrelated class	outside_canary	0	Not applicable

5.7 Hostile refusal as negative competence

Grand Closure G6 recorded 12 hostile cases and reported all_refused_correctly=true. Cases included capability replay, stale process lease, ARDA refusal, wrong peer identity, missing kernel seals, policy drift, source-state drift, workspace escape, file-descriptor substitution, revoked promotion, and additional integrity attacks. Refused cases recorded zero physical effects and no residue. The observed refusal categories matched the expected categories. This evidence operationalises negative competence: the capability boundary contains reasons not to act, and refusal is a successful verified outcome.

5.8 Proof summary

Finding	Observed evidence	Claim boundary
Verified recurrence	24/24 governed hidden passes versus 2/24 comparison; $p < .001$ in paired slice.	Bounded six-family mature NVIDIA slice.
Cloud-call displacement	12 fewer, 12 tied, 0 more; QPCCD 0.50 [0.314, 0.686].	Does not establish general realised savings.
Evidence threshold	All six families first reached zero-call recurrence at O3.	Policy-specific, not a universal sample-complexity law.
Provider independence	12/12 cross-provider reuse; no target-provider execution.	Harnessed repository-bound capabilities.
Mutation safety	6/6 stale blocks; 6/6 verified recovery; 0 false reuse; 24/24 correct ladder decisions.	Controlled mutation tiers.
Crystal-only path	6/6 held-out proposals, reproduced; zero generated tokens.	Proposal-only bounded tasks.
Capsule enforcement	Required capsule succeeds lawfully; missing/unverified capsules refuse without fallback.	One canary task class.
Hostile refusal	12/12 correct refusals with zero physical effects.	Grand Closure hostile matrix, not universal adversarial security.

6. Discussion

6.1 A computational category between memory and software

PGEC identifies a category not adequately described by cache, memory, fine-tuning, or conventional software. A cache returns prior material because identity or similarity conditions match. Memory provides evidence or context to another inference. Fine-tuning changes parametric behaviour. Conventional software is explicitly authored in advance. A proof-governed compiled capability is experience-derived, executable, authority-bounded, and continuously conditional on external proof.

Mechanism	Reusable unit	Primary gate	Execution authority
Text/semantic cache	Prior answer or embedding-neighbour result	Identity or similarity	Usually external to cache
KV/prefix cache	Engine-specific token state	Prefix and engine compatibility	Model serving runtime
Agent memory	Prior transcript, summary, or fact	Retrieval relevance	None by itself
Fine-tuning/adaptor	Changed parametric behaviour	Training and evaluation	Model runtime, often broad
Conventional program	Authored algorithm	Input contract and testing	Program permissions
PGEC capability	Verified causal/behavioural episode compiled into bounded execution	Applicability, authority, verifier, policy, state fingerprints	Explicit maximum envelope; promotion remains separate

6.2 Verified effects, not linguistic resemblance

The experiments support a shift from textual equivalence to behavioural admissibility. Similar prompts may refer to different repository states, policies, tools, or postconditions. Dissimilar prompts may request the same bounded transformation. PGEC therefore treats language as one clue, not the execution licence. Reuse is governed by state fingerprints, causal structure, verifier requirements, and effects.

6.3 Refusal is constitutive intelligence

A system that can execute but cannot delimit its own applicability is not a governed capability. The mutation and hostile-refusal evidence shows that non-execution can be the correct and economically rational outcome. This reframes benchmark design: a capability should be evaluated not only on successful transformations but also on correct refusal, escalation, revocation, and absence of residue.

6.4 Causal evidence without hidden chain of thought

PGEC does not require the system to preserve private model reasoning or treat self-explanation as faithful causality. It compiles observable state transitions, attributed process and tool actions, file and socket effects, authority records, and independent verifier outcomes. This supports accountability while avoiding a misleading equation between narrated reasoning and actual causal mechanism.

6.5 Authority-preserving learning

The system becomes more capable without automatically becoming more powerful. A promoted capability can remain narrower than the model-mediated episode from which it originated. Its authority can be restricted by path, task class, target, operation, duration, and one-use capability. This creates a form of learning in which competence can accumulate while authority remains explicitly bounded.

6.6 Inference economy inversion as consequence

The earlier inference-economy inversion becomes more precise under PGEC. The economic transformation is not caused by storing outputs alone. It is caused by compiling verified experience into a lower-entropy local execution path whose lookup and verification costs are lower than renewed inference, while safety remains equal or stronger. Inference is therefore spent where uncertainty remains and converted into governed infrastructure where proof supports recurrence.

6.7 Intelligence accumulation without weight change

The crystal-only results demonstrate a bounded way in which system capability can improve without modifying model weights. The surrounding runtime accumulates verified transforms, refusal boundaries, route knowledge, and verifier-linked capabilities. This does not imply general intelligence gain. It shows that effective system competence can compound through non-parametric, proof-governed assets.

7. Threats to Validity and Limitations

- The strongest inferential statistics apply only to the 24-pair mature NVIDIA slice. The wider 198-record package is staged and non-IID.
- The designed 450-observation core matrix remains incomplete, so task and provider generalisation cannot be claimed.
- Many experiments use bounded synthetic or harnessed task families. Natural, uncontrolled coding-session crystallisation remains an empirical frontier.
- The crystal-only route demonstrates bounded proposal recurrence, not arbitrary multi-file autonomous coding competence.
- Cross-provider consumption shows route independence in the harness, not universal portability across repositories, organisations, hosts, or policy domains.
- The hostile matrix is substantial negative evidence but cannot prove immunity to all attacks.
- The capsule-required result applies to one canary class and should not be generalised to every execution path until wider closure tests pass.
- Estimated avoided tokens and costs are not equivalent to audited realised production savings.
- The available LoRA evidence does not establish promoted Qwen weight-level performance improvement; the incompatible adapter result is governance evidence only.
- Raw provider response bodies were not retained in portions of the older evidence package, limiting forensic reproduction of provider output.
- The paper synthesises a rapidly evolving research build. A generated release identity and single canonical evidence ledger remain necessary before archival publication.

8. Implications and Future Work

8.1 Immediate empirical programme

1. Close the canonical AgentRun path and demonstrate one uninterrupted inspect-edit-test-repair-SourcePlan mission with restart, cancellation, and post-promotion verification.
2. Complete the 450-observation controlled matrix with preregistered task families, routes, occurrence points, and lanes.
3. Collect naturalistic episodes from ordinary development sessions and test whether candidate extraction discovers useful invariants without synthetic repetition.
4. Evaluate compound experiential compilation in which several independently verified capabilities compose through typed dependencies and each stage may refuse independently.

5. Measure total recurrence economics, including lookup, verifier, maintenance, false-hit, revalidation, and operator costs rather than token savings alone.
6. Test cross-host and cross-policy portability using capability cards, attestation, sealed capsules, and explicit environmental compatibility proofs.
7. Develop public PGEC benchmark tasks that reward correct refusal and zero residue, not merely successful execution.

8.2 Broader research questions

Future work should investigate how much causal structure can be inferred conservatively from observed episodes, how applicability boundaries should decay, whether proof density predicts safe generalisation, how capability composition affects risk, and whether capability libraries can be exchanged without transferring hidden authority or private evidence.

9. Conclusion

This paper introduced proof-governed experiential compilation as a systems mechanism for transforming verified agentic experience into bounded reusable compute. PGEC treats the durable runtime episode, not the prompt or response, as the unit of experience. It compiles observable state, attributable action, consumed authority, verifier outcomes, and physical effects into a candidate capability with explicit preconditions, parameters, postconditions, applicability, authority, and revocation rules.

The supplied BEAST evidence provides bounded but unusual proof of mechanism: verified recurrence with lower cloud-call use, consistent evidence-triggered crystallisation across six task families, provider-independent consumption, mutation-sensitive refusal and recovery, zero-token crystal-only held-out recurrence, failed-closed adapter behaviour, mandatory capsule enforcement for a canary class, and correct zero-effect refusal across a hostile matrix.

The contribution is not that prior outputs can be cached. It is that successful agentic experience can leave behind an executable and governable residue whose authority is narrower than the original agent and whose validity remains conditional on continuing proof. Inference economy inversion is the economic consequence of this mechanism: probabilistic compute is spent for unresolved uncertainty, then preserved as local capability only where evidence continues to justify reuse.

In its most compact form:

$$experience \rightarrow evidence \rightarrow bounded\ program \rightarrow verified\ recurrence\ or\ refusal$$

References

- [1] SWE-bench Verified. Official benchmark description.
- [2] OpenAI. Structured Outputs documentation.
- [3] Model Context Protocol. Specification 2025-06-18.
- [4] International Energy Agency. Energy and AI: Energy demand from AI.
- [5] Hugging Face. Text Generation Inference documentation.
- [6] vLLM. Automatic Prefix Caching design documentation.
- [7] Cheng, Y. et al. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv, 2025.
- [8] Ong, I. et al. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665, 2024.
- [9] Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.

[10] Bunt, B. BEAST Inference Economy Inversion Working Paper v0.9: Verified Compute Commons and Final Runtime Layer. Claim-bounded preprint, 2026.

[11] Bunt, B. EdgeK-BEAST full research build and evidence archive, supplied 20 July 2026.

Appendix A. Evidence and Implementation Anchors

The following paths identify the principal source and test artifacts used to ground this initial draft. Paths are relative to the supplied BEAST full-build archive.

Evidence purpose	Representative archive paths
AgentRun runtime	app/kernel/agents/run_engine.py; run_store.py; run_events.py; run_cancel.py; planner_runtime.py; tool_runtime.py
AgentRun tests	tests/test_agent_run_store.py; tests/test_agent_planner_runtime.py; tests/routes/test_agent_run_routes.py; tests/routes/test_agent_run_execution.py
Worktree and promotion	app/kernel/workspaces/worktree_forge.py; app/kernel/agents/promotion_engine.py; app/kernel/evidence/sourceplan_handoff.py; tests/phase2/test_worktree_mutation_tools.py; tests/phase3/test_sourceplan_handoff.py
Evidence and compatibility	app/kernel/evidence/evidence_ledger.py; fingerprint_engine.py; compatibility_engine.py; equivalence_engine.py; tests/phase3/test_evidence_ledger.py
Crystallization	app/kernel/compute/runtime_crystallizer.py; crystal_generalizer.py; governed_crystal_executor.py; crystal_reuse_gateway.py; typed_crystal_ir.py
Crystallization tests	tests/test_runtime_crystallizer.py; test_crystal_generalizer.py; test_governed_crystal_executor.py; test_typed_crystal_interpreter.py; test_compound_crystallization.py
Sensorium	app/kernel/compute/capsule_sensorium.py; unified_evidence_packet.py; tests/test_sensorium_runtime.py; test_sensorium_physical_effects.py; test_sensorium_crystallization.py
Mature inference evidence	benchmarks/results/beast_definitive_mega_test_master_evidence/ and source runs; v0.9 paper Tables 6-8
Mutation evidence	benchmarks/results/beast_definitive_mega_test_master_evidence/ source_runs/definitive_mega/ beast_definitive_mega_test_nvidia_mutation_o10/
Capsule closure	evidence/grand_closure/g8-capsule-required-1784543526077940090.json
Hostile refusal	evidence/grand_closure/grand_closure_g6_hostile_refusal_20260720T103205Z.json

Appendix B. Claim Ledger

Claim	Status in this draft
Bounded verified recurrence can displace some later provider calls.	Supported in mature paired slice and cross-provider harness.
A promoted capability can remain provider-independent.	Supported for 12 harnessed cross-provider cases.
Material drift can trigger safe block and fresh recovery.	Supported for six live recovery cases and 24 mutation-tier

	decisions.
Correct refusal can produce zero physical effects.	Supported in 12-case hostile closure and capsule-required refusals.
PGEC works for arbitrary natural coding tasks.	Not proven.
General multi-crystal composition is safe and useful.	Emerging implementation; not yet established by broad evidence.
Local adapters generally improve coding capability.	Not proven; incompatible adapter failed closed.
BEAST produces general realised production savings.	Not proven.
Compiled capabilities are universally portable across hosts.	Not proven.